

EE 533 – Final Project (Spring 2026)

Note: Undergraduate students will work in groups of three and complete Tasks 1-7.

Graduate students will work in pairs and complete Tasks 1-8.

Overview

In this project, you will design an analog integrate-and-fire (I&F) neuron circuit and use it to inform the behavior of a spiking neural network (SNN). You will bridge circuit-level design and SNNs by embedding your circuit's characteristics into a Python-based SNN and validating consistency between circuit simulations and software models. This project requires you to design and characterize a transistor-level neuron circuit, extract its behavior and reproduce it in software, use the circuit-informed neuron in an SNN, validate consistency between circuit and software, and evaluate system-level performance under realistic neuron constraints.

Tasks

1. Design an integrate-and-fire (I&F) neuron circuit in Cadence that satisfies:
 - a. Input current range: 1 nA to 100 nA
 - b. A linearly proportional firing rate over approximately two decades (e.g., 1 Hz–100 Hz, 100 Hz–10 kHz, or similar ranges)
 - c. Demonstrated input current vs. firing frequency (f-I) characteristic
2. Circuit-Informed Neuron Model in Python. Develop a neuron model in Python that matches your circuit behavior. This model must:
 - a. Reproduce the measured f-I relationship
 - b. Capture key dynamics (integration, threshold, reset)
 - c. Replace the default neuron model in your SNN framework (e.g., SpikingJelly)
3. Single-Neuron Validation (Circuit vs Python). Validate consistency between circuit and software. Apply identical inputs to both:
 - a. Constant current
 - b. Pulsed currentCompare and provide quantitative agreement metrics and plots in terms of:
 - a. Spike timing
 - b. Firing rate
4. SNN Design and Training. Using your circuit-informed neuron, train an SNN on MNIST. The architecture should be an input layer, a single hidden layer (size selectable: 512, 256, 128, 64, 32 neurons), and an output layer (10 neurons). Train and evaluate using:
 - a. Classification accuracy.
5. Partial Hardware Validation (4-Neuron Circuit). This step verifies circuit-to-network consistency. To validate the network at circuit level select only four neurons (any four of the neurons) from the hidden layer. Using trained weights:
 - a. Generate input current waveforms (PWLF format)

- b. Feed them into four neuron circuit instances in Cadence Run transient simulation
 - c. Compare for each neuron (Cadence vs Python) the following:
 - i. Spike timing
 - ii. Temporal alignment
- 6. Default Neuron vs. Circuit-Informed Neuron. Compare SNN performance using the default neuron model and your circuit-informed neuron model. Evaluate *classification accuracy* across the following parameters:
 - a. Image Resolution: 4×4, 7×7, 14×14, 28×28
 - b. Temporal Encoding: 5 time steps, 10 time steps, 20 time steps
 - c. Training Method: Surrogate gradient descent (supervised), STDP (unsupervised)
 - d. Weight Quantization: 2-bit, 4-bit, 8-bit
 - e. Analyze how each factor affects the accuracy.
- 7. Network Simplification. Reduce model complexity while maintaining performance. Decrease
 - a. Hidden layer size
 - b. Time steps
 - c. Weight precision
 - d. Input resolutionWhile still reaching the target of >75% accuracy
- 8. (Graduate students only). Graduate students will extend the circuit validation to a larger system:
 - a. Multi-Neuron Circuit Simulation (32-Neuron Hidden Layer). Expand the neuron circuit to 32 instances using netlist-level replication. The SNN should perform a simpler binary classification task (digits 0 vs 1)
 - b. Workflow:
 - i. Generate input current waveforms from trained Python SNN
 - ii. Use netlist-based simulation (Cadence + OCEAN scripting)
 - iii. Run inference on 10 test images (5 per class)
 - iv. Export neuron outputs to CSV
 - v. Extract spike timing
 - c. Compare:
 - i. Circuit spike timing vs Python hidden-layer spikes
 - ii. Agreement across all neurons

Report Requirements (20 points)

(Report submission deadline: Monday May 4th, 11:59PM.)

The final report must be clear with plots and tables in the report, and codes in the appendix. Follow the following report structure using the IEEE Conference Template available:

<https://www.ieee.org/conferences/publishing/templates>

Page limit: 6 pages for all of the sections below.

1. Introduction
2. Methodology
3. Results
4. Conclusion
5. Appendix
6. References
7. Author Contributions: Each student must provide a brief description of their contributions (approximately two sentences per student).

Final (Flash) Presentation Requirements (10 points)

(Slide submission deadline: Sunday May 3rd, 11:59PM.)

(Presentation: Monday, May 4th, 5:30PM-6:45PM)

Each team will deliver a brief, time-limited flash presentation summarizing their project results:

- a. **3-minute** per team (strictly enforced)
- b. This presentation is **not** a summary of your full report. Instead, focus on presenting your best-performing results
- c. Be concise and use visuals while highlighting the key points.

References

[1] Fang, Wei, Yanqi Chen, Jianhao Ding, Zhaofei Yu, Timothée Masquelier, Ding Chen, Liwei Huang, Huihui Zhou, Guoqi Li, and Yonghong Tian. "Spikingjelly: An open-source machine learning infrastructure platform for spike-based intelligence." Science Advances 9, no. 40 (2023): eadi1480.

[2] Fang, W., SpikingJelly: An open-source machine learning framework for spiking neural networks. GitHub. <https://github.com/fangwei123456/spikingjelly>